



資料結構

Data Structure

Lab 11

姓名：林彥廷

學號：112370219

Lab11-Q1.

計算 level sum:

依照 BFS 邏輯,並根據輸入的層數,計算該層節點數數值總和,如果輸入數值超過樹高則需顯示提示。

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
const int EMPTY = NULL; // 用 NULL 代表沒有節點
class TreeNode {
public:
    int value; // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty() && i < arr.size()) {
            TreeNode** nodePtr = q.front(); // 取出 queue 的第一個節點指標
            q.pop(); // 將該節點從 queue 中刪除
```

```

        // 左子節點
        if (i < arr.size()) {
            if (arr[i] != EMPTY) {
                (*nodePtr)->left = new TreeNode(arr[i]); // 添加左子節點
                q.push(&((*nodePtr)->left)); // 將左子節點的指標加入 queue
            }
            i++;
        }
        // 右子節點
        if (i < arr.size()) {
            if (arr[i] != EMPTY) {
                (*nodePtr)->right = new TreeNode(arr[i]); // 添加右子節點
                q.push(&((*nodePtr)->right)); // 將右子節點的指標加入 queue
            }
            i++;
        }
    }
    return root;
}

void Depth_first_search(TreeNode* node) { // 深度優先搜尋
    if (node == nullptr) return;
    cout << node->value << " "; // 訪問當前節點的值
    Depth_first_search(node->left); // 遞迴遍歷左子樹
    Depth_first_search(node->right); // 遞迴遍歷右子樹
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
    q.push(root); // 將根節點的指標加入 queue

    while (!q.empty()) {
        TreeNode* current = q.front(); // 取出 queue 的第一個節點指標
        q.pop(); // 將該節點從 queue 中刪除
        cout << current->value << " ";
        if (current->left) q.push(current->left); // 將左子節點的指標加入
queue
    }
}

```

```

        if (current->right) q.push(current->right); // 將右子節點的指標加入
queue
    }
}

void Level_sum(TreeNode* root, int layer) {
    int level = 0;

    queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
    q.push(root); // 將根節點的指標加入 queue

    while (not q.empty()) {
        int nodeCount = q.size(); // 計算該層有多少節點
        int sum = 0; // 計算總和

        for (int i=0; i<nodeCount; i++) { // 將該層節點 pop 出來作加總
            TreeNode* node = q.front();
            q.pop();
            sum += node->value; // 將目前節點值存入總和

            // 將左右子節點 push 進 queue ，當這層結束時，queue 裡就是下一層節點
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }

        if (level == layer) { // 如果 level 等於用戶要找的層，就輸出總和
            cout << "The sum of level " << level << " is: " << sum << endl;
            return;
        }
        level++; // 進入下一層
    }
    cout << "The layer exceeds the tree height." << endl; // 輸出的層超出樹高
};

int main() {
    int layer;
    BinaryTree tree;

```

```

    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL,
    NULL };
    tree.buildTree(arr);

    cout << "DFS Result: ";
    tree.Depth_first_search(tree.root);
    cout << endl;

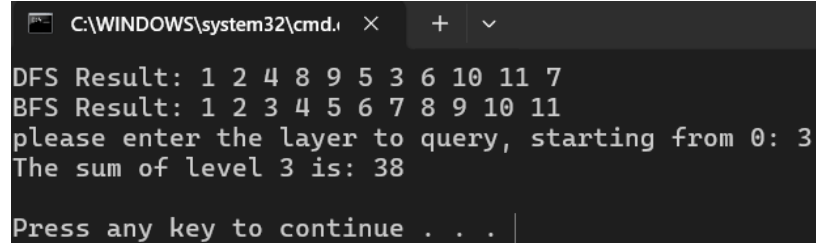
    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;

    cout << "please enter the layer to query, starting from 0: ";
    cin >> layer;
    tree.Level_sum(tree.root, layer);

    return 0;
}

```

Result



```

C:\WINDOWS\system32\cmd.exe
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0: 3
The sum of level 3 is: 38

Press any key to continue . . . |

```

```
Lab11_Q1.cpp 1,1 U X
Lab11 > C:\Lab11_Q1.cpp > main()
15 class Binarytree {
59
60 void Breadth_first_search(TreeNode* root) {
61     if (root == nullptr) return;
62     queue<TreeNode*> q; //建立queue儲存待處理的節點指標
63     q.push(root); // 將根節點的指標加入queue
64
65     while (!q.empty()) {
66         TreeNode* current = q.front(); // 取出queue的第一個節點指標
67         q.pop(); // 將該節點從queue中刪除
68         cout << current->value << " ";
69         if (current->left) q.push(current->left); // 將左子節點的指標加入queue
70         if (current->right) q.push(current->right); // 將右子節點的指標加入queue
71     }
72 }
73
74 void Level_sum(TreeNode* root, int layer) {
75     int level = 0;
76
77     queue<TreeNode*> q; // 建立queue儲存待處理的節點指標
78     q.push(root); // 將根節點的指標加入queue
79
80     while (not q.empty()) {
81         int nodeCount = q.size(); // 計算該層有多少節點
82         int sum = 0; // 計算總和
83
84         for (int i=0; i<nodeCount; i++) { // 將該層節點pop 出來作加總
85             TreeNode* node = q.front();
86             q.pop();
87             sum += node->value; // 將目前節點值存入總和
88
89             // 將左右子節點push 進queue，當這層結束時，queue 裡就是下一層節點
90             if (node->left) q.push(node->left);
91             if (node->right) q.push(node->right);
92         }
93     }
94 }
```

```
C:\WINDOWS\system32\cmd. x + v
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0: 3
The sum of level 3 is: 38
Press any key to continue . . . |
```

```
C:\WINDOWS\system32\cmd. x + v
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0: 5
The layer exceeds the tree height.
Press any key to continue . . . |
```

```
Lab11_Q1.cpp 1,1 U X
Lab11 > C:\Lab11_Q1.cpp > main()
15 class Binarytree {
59
60 void Breadth_first_search(TreeNode* root) {
61     if (root == nullptr) return;
62     queue<TreeNode*> q; //建立queue儲存待處理的節點指標
63     q.push(root); // 將根節點的指標加入queue
64
65     while (!q.empty()) {
66         TreeNode* current = q.front(); // 取出queue的第一個節點指標
67         q.pop(); // 將該節點從queue中刪除
68         cout << current->value << " ";
69         if (current->left) q.push(current->left); // 將左子節點的指標加入queue
70         if (current->right) q.push(current->right); // 將右子節點的指標加入queue
71     }
72 }
73
74 void Level_sum(TreeNode* root, int layer) {
75     int level = 0;
76
77     queue<TreeNode*> q; // 建立queue儲存待處理的節點指標
78     q.push(root); // 將根節點的指標加入queue
79
80     while (not q.empty()) {
81         int nodeCount = q.size(); // 計算該層有多少節點
82         int sum = 0; // 計算總和
83
84         for (int i=0; i<nodeCount; i++) { // 將該層節點pop 出來作加總
85             TreeNode* node = q.front();
86             q.pop();
87             sum += node->value; // 將目前節點值存入總和
88
89             // 將左右子節點push 進queue，當這層結束時，queue 裡就是下一層節點
90             if (node->left) q.push(node->left);
91             if (node->right) q.push(node->right);
92         }
93     }
94 }
```

```
C:\WINDOWS\system32\cmd. x + v
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0: 5
The layer exceeds the tree height.
Press any key to continue . . . |
```